actor.java

java

1793219890427_MainVascular.java

java

1793219890428_NullToNoInformationDeserializer.java

java

1793219890429_ValueLocator.java

java

1793219890430_VascularDbImporter.java

java

1793219890430_VascularModels.java

java

1793219890431_VascularRunner.java

java

1793219890432_VascularThroughputHarness.java

java

1793219890433_VascularTsvWriter.java

java

I am building a Java application using the Eclipse IDE. The application is designed to extract arterial flow velocities, percentage of stenosis, and internal to common carotid artery ratios from radiology reports on vascular flow in the supplying vessels of the head. The reports are from ultrasound-generated reports in the VA on Veterans with suspected vascular disease.

We are planning to use a quantized large language model, Gemma 4, that is hosted by a llamacpp application running locally. Currently, we are running the application on a Intel Silver 4310 CPU with 256 Gb of RAM and a Nvidia A2 GPU. We are starting the llamacpp application with the following command:

start "" "P:\ORD_Nguyen_202206062D\Glenn\LlamaAppInstall_261008\llama-b11514-bin-win-cuda-12.4-x64\llama-server.exe" --host 127.0.0.1 --port 8080 --model "O:\OMHSP_CONDUIT\NLP\Glenn\LLMs\Models\gemma-4-E4B-it-IQ4_NL\gemma-4-E4B-it-IQ4_NL.gguf" --n-gpu-layers 999 --ctx-size 49152 --parallel 4 --cont-batching --batch-size 2048 --ubatch-size 512 --flash-attn on --threads 12 --load-mode mlock

I have pasted in the Java code we are using in this prompt, and I would like you to review that code. Then I will ask you to make changes to the code to address some additional needs.

Show more

Read 9 files

Read 9 files

Let me check the uploads directory in the sandbox.

Ran 4 commands

Ran 4 commands

I read all nine files. The architecture is sound: a system/user prompt split for KV-cache reuse, a JSON schema constraint, a truncation-aware retry, per-document failure isolation, and a locator that finds offsets in Java instead of trusting the model. Below are the problems I'd fix, roughly in priority order.

Accuracy risks

1. **Vertebral and subclavian vessels have no place in the schema.** Carotid duplex reports almost always include vertebral artery velocities and direction.
   The vessel enum only allows ICA, ECA and CCA, and the prompt never says to skip other vessels. The model will either mislabel a vertebral value as ICA or CCA, or drop it unpredictably. Either add VERTEBRAL or Other to the enum, or add an explicit "ignore all other vessels" rule. Also, vessel_location has no "bulb" or "bifurcation" option, which reports use often.
2. **Forced defaults hide missing data.** If the report doesn't say PSV or EDV, the model must write PSV. If it doesn't state flow direction, the model must write anterograde. Downstream, an unstated value is indistinguishable from a stated one. I'd add Unspecified to both enums, in the schema, prompt, repair prompt and validate(). You can then decide in analysis how to treat those rows.

3. **Reports often state the same finding twice.** Stenosis commonly appears in both the body and the Impression. Nothing deduplicates these, so you may get double counts. Decide on a policy, such as keeping both and tagging the section, or collapsing duplicates.
4. **Units are dropped silently.** If a report ever gives m/s, the value is stored without a flag. This is low risk, but a sanity check on plausible ranges would catch it.

Robustness bugs

1. **Files.readString(path) assumes UTF-8.** VA text exports containing Windows-1252 characters (degree signs, "µ", smart quotes) throw MalformedInputException, and the document fails. Read the bytes and decode with a fallback charset. Offsets then stay consistent with the text you pass to the model.
2. **There is no context-fit guard.** Each slot holds 12,288 tokens, and you reserve 7,168 for output. Any prompt over about 5K tokens (a long multi-study report) overflows. The retry at 8,192 would then fail the same way. Count tokens (llama-server has /tokenize, or estimate from character count) and lower max_tokens for long documents. Flag documents that can't fit.
3. **The HTTP request has no timeout.** If the server stalls, that worker thread hangs forever, and analyzeDocumentsParallel never returns. Add .timeout(...) to the request and a connect timeout on the client. I'd also pin HttpClient.Version.HTTP_1_1, since llama.cpp doesn't speak HTTP/2 and the default client tries an h2c upgrade first.
4. **Runaway generation is a plausible cause of your 4,300–4,800 token outputs.** Greedy decoding with a repeat penalty of 1.0 is prone to repetition loops in small models. A typical report needs about 1,000 tokens (your example with 13 velocities is roughly that). Check a few of those long outputs for repeated entries. If you find loops, add duplicate detection and consider streaming so you can abort early. Also check whether Gemma 4's chat template is emitting thinking or reasoning tokens. If so, disabling them would cut generation time. I can't verify that for this build.
5. **The locator's cursor assumes document order.** It picks the first candidate after the previous value's start offset. If the model groups output by vessel while the report is organized by side, repeated excerpts like CCA: PSV 82 cm/s can resolve to the wrong occurrence. Tracking a set of already-claimed offsets, instead of only a monotonic cursor, would be more robust.

6. **Resume and rerun are weak.** Output is append-only, so rerunning duplicates records. The _failed.txt list is written, but nothing reads it back. Add a "process only these paths" option and skip documents already present in the output.
7. **Zero-finding documents leave no trace in the TSVs.** You can't tell "processed, nothing found" from "failed" without the log. A fourth documents.tsv with status, counts and match statistics would fix that.

Smaller issues

- VascularRunner runs the whole extraction on the Swing event-dispatch thread. An uncaught exception, such as a bad argument to RunType.valueOf, skips System.exit(0) and leaves the JVM hanging. It also uses scanner.next(), which breaks on paths containing spaces.
- getOutputFile() closes System.in through try-with-resources on Scanner. This works today but will break any later console input.
- estimateRemainingProcessingTime divides by zero if elapsed time is 0 ms.
- The DB importer doesn't store source_text, offsets or match method, so the database is less informative than the TSVs. It also has no duplicate protection on reimport, and it calls System.exit inside a library method.
- Several comments and usage messages still refer to the …Revised class names.
- ExecutorService as an AutoCloseable needs Java 19 or later, so check the Eclipse compliance level.
- In the throughput harness, a 20-document sample with 4 threads is skewed by tail effects, since the last documents run with idle slots. The warm-up document is also in the timed sample. Capturing llama.cpp's timings (tokens per second) from each response would give you cleaner numbers.

Server command

Everything in the launch line is sensible for a 16 GB A2 with all layers on the GPU. --mlock matters little when weights live in VRAM. I can't verify --load-mode mlock against build b11514. The older spelling is plain --mlock, so confirm with llama-server --help that the flag is actually taking effect.

Which changes would you like me to make first?

Let's go through these issues one at a time.

Regarding "Accuracy risks"

1. Please add an explicit "ignore all other vessels" rule, and include "bifurcation" and bulb as vessel location options.
2. Please add "Unspecified" to the PSV or EDV and the anterograde or retrograde options.
3. Continue to keep duplicates. Do not collapse them.
4. Please add the units as a separate item to extract within flow_velocities. The options for units should be "cm/s", "m/s", "Other", or "Unspecified."

Thought process

Thought process

I'll copy the five affected files into my working directory and apply the edits there. The originals stay untouched.

Ran 6 commands, read a file

Ran 6 commands, read a file

I made all four changes in five files: MainVascular, LlamaVascularExtractor, VascularModels, VascularTsvWriter and VascularDbImporter. I couldn't compile them here because the sandbox has no javac. I checked each change by diffing against your originals, so please run a build in Eclipse. The other four files (ValueLocator, VascularRunner, VascularThroughputHarness, NullToNoInformationDeserializer) are unchanged.

1. Other vessels and new locations

- **Other vessels:** The system prompt now says to extract only ICA, ECA and CCA. It tells the model to ignore the vertebral, subclavian, innominate, basilar and intracranial arteries, and not to relabel their values as ICA, ECA or CCA.
- **Few-shot example:** I added a vertebral artery line to the example report. It has no matching entry in the example output, so the model sees the ignore rule applied.
- **Locations:** vessel_location now allows bulb and bifurcation. I updated this in the prompt, the JSON schema, the repair prompt, and a new location check in validate().

2. "Unspecified" for velocity type and flow direction

- **Enums:** velocity_type is now PSV, EDV or Unspecified. flow_direction is now anterograde, retrograde or Unspecified.
- **Prompt rules:** The old "default to PSV" and "default to anterograde" rules are gone. Retrograde is used only when the report says so. Anterograde is now used only when the report explicitly says antegrade or anterograde. Everything else is Unspecified.
- **Example output:** Every flow direction in it changed from anterograde to Unspecified, because that example report never states direction. Leaving it unchanged would have taught the model to keep guessing.
- **NoMeasurableFlow entries:** They now use the stated velocity type, or Unspecified, instead of forcing PSV.

3. Duplicates

- Nothing changed, and nothing in the pipeline removes duplicates. The prompt doesn't ask the model to collapse them. I can add an explicit rule to report every occurrence separately if you see the model skipping repeats.

4. Units

- **New field:** flow_velocities has a new units field with the options cm/s, m/s, Other and Unspecified. It sits right after value in the prompt schema, JSON schema, repair prompt, both model records, validate() and the console summary.
- **Prompt rule:** The prompt tells the model to record the unit as stated, never convert it, and keep it out of value. Every velocity in the example output now carries "units":"cm/s".
- **TSV:** units is a new column after value in the flow velocities file.
- **Database:** The importer now writes Units as the eighth parameter of the velocity insert.

Before you run anything, note two things.

- **Database column:** ggLLM.FlowVelocity.Units must exist as a nullable column, such as NVARCHAR(50). I updated the header comment in the importer with the ALTER TABLE note.
- **Mixed TSV files:** If you append to a flow velocities file made by the old version, its header and rows won't match the new column layout. Start a new output file for the next run.

Which accuracy issue would you like next, or should I move on to the robustness bugs?

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\MainVascular.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaVascularExtractor.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularModels.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularTsvWriter.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularDbImporter.java

Code

Thought process

Thought process

Here is how I'd structure it. The main idea is to treat each test as a recorded experiment run instead of a one-off launch. Each run gets its own folder holding the results, the exact conditions and the timing data. The comparisons then become joins on run ID and document ID, and you can do them in your own application.

1. Define what a "run" is

A run is one combination of machine, model, server configuration, prompt version and document list. The code should record all of these automatically, so you never have to

remember what produced a file. I'd create a run folder for each run, named something like run_<timestamp>_<machine>_<model>_<label>. It would contain:

- **manifest.json** with everything needed to reproduce the run:
    - hostname, OS, CPU, RAM, GPU and VRAM;
    - the llama.cpp build (parsed from the server log);
    - the model file name, size and ideally a hash;
    - the full resolved server command and all settings;
    - the prompt file name and its SHA-256;
    - the sampling parameters;
    - the application version (a git commit if you have one);
    - the hash of the document list.
- **results.json**, the extraction output you already produce.
- **documents.tsv**, extended with per-document timing and token counts (see section 3).
- **summary.json**, the aggregate speed figures.
- **server.log** and **raw_responses/**, which holds the model's raw text for every document before parsing or repair. When you score accuracy, this lets you tell a parse or schema failure apart from a genuine extraction error.

A single runs.csv, with one appended row per run, gives you the comparison table across everything.

2. Separate hardware settings from experiment settings

Right now one properties file mixes both. I'd split it into two layers:

- The **machine file** holds what's fixed per computer: executable path, GPU layers, threads, flash-attn and corpus location. You already have one per computer.
- An **experiment file** holds what you are testing: model path, prompt file, ctx-size, parallel, cache types, sampling, MAX_TOKENS and the label. It overrides the machine file.

Launching would then take both, for example --config=machine.properties --experiment=gemma4-e4b-promptB.properties.

A batch runner can go one step further. It would read an **experiment matrix** (a list of experiment files or CSV rows) and run them one after another, starting and stopping the server each time. It would continue past failures and write each run to its own folder. That lets you queue a day's worth of runs on one machine and leave it. The batch mode must be fully non-interactive, so all the console prompts (output file, resume, parallel count) come from config instead.

3. Move the accuracy-relevant items out of code

Prompt testing needs the prompt in an external file, which is currently a constant in code. These items would need to become settings:

- The **system prompt** and the example, as a file with an ID and hash. Ideally the JSON schema or grammar can also be varied, since that's a form of prompt engineering.
- **Sampling:** temperature, top_p, top_k and seed. Use temperature 0 and a fixed seed for comparisons.
- **MAX_TOKENS**, the thinking on/off switch, and the schema-versus-grammar switch. The thinking workaround is Gemma-specific, so other model families need their own handling.
- **Chars per token** for the context budget. Tokenizers differ by model, so the 3.0 estimate may be wrong for others. The extractor already collects real usage stats, so you can calibrate it per model.
- Any **chat-template** or --jinja flags the new model needs. Models differ in how they handle the system role and response_format, so expect a small per-model adapter section.

4. Capture speed data that is comparable across machines

Total wall time alone isn't enough, because it mixes several effects. I'd record:

- **Server start and model load time**, separately from processing time.
- **Per document:** a stable document ID (the file name, not the full path, since paths differ by machine), queue and request latency, prompt tokens, generated tokens, retry or repair flags, and status.
- **llama-server's own timings**, which it returns in each response: prompt tokens per second and generation tokens per second. These let you compare machines independently of how the work was batched.

- **Aggregates:** documents per minute, median and 95th-percentile latency, and failed or too-long counts.
- **Optional resource sampling:** GPU use, VRAM and RAM every few seconds into a CSV. This can be a small external script (nvidia-smi on the Xeon, Activity Monitor or powermetrics on the Mac) rather than application code.

5. Keep the comparisons valid

- **Freeze the 100 reports.** Use one document-list file and record its hash. The list-file option already exists, so use it instead of scanning the directory. Confirm both computers hold identical files.
- **Use the same document order, warm-up and parallel setting** when you are comparing speed.
- **Repeat each run** (three times is a reasonable minimum) and compare the spread. Parallel batching can make output slightly non-deterministic even at temperature 0, so one run isn't enough for either speed or accuracy.
- **Disable resume** or use a fresh run folder every time, so a run never skips documents already done.
- **Vary one factor at a time.** I'd go in this order:
    1. Fix the model and prompt, then vary server settings (parallel, threads, batch sizes, GPU layers, flash-attn) to measure speed. These rarely change accuracy.
    2. Fix the server settings, then vary the prompt to measure accuracy and its speed cost. A longer prompt and longer outputs both slow things down.
    3. Then vary the model and quantization.
    - The exceptions to step 1 are KV-cache quantization (--cache-type-k/v), --ctx-size (documents get skipped) and model quantization. These can change accuracy, so check them against your reference set too.
- **Make comparisons paired.** With only 100 reports, comparing per-document results across runs is far more informative than comparing overall averages.

6. For your accuracy comparison

The existing output already carries what a reference-standard comparison needs: source_text, offsets and a document identifier. I'd only add a **stable document ID**

and run ID on every record and keep finding order consistent. Duplicates should stay uncollapsed, as you've specified. Your comparison tool can then join results to your manual reference by document ID and group by run.

7. Handling of PHI

The run folders will contain report excerpts in source_text and in the raw responses. Keep them on approved VA storage. Before copying results from one machine to another, confirm that both are authorized for this data, especially the Mac.

Suggested order of work

1. Run folder, manifest and runs.csv, plus per-document timing and token telemetry. This gives the most value for the least change.
2. Externalize the prompt, sampling and MAX_TOKENS into the experiment file, with the two-layer config.
3. Non-interactive batch runner driven by an experiment matrix. It would generalize your current throughput harness, which only sweeps the parallel count.
4. Per-model adapter settings (tokenizer ratio, chat template, thinking handling) as you add models.
5. Optional resource sampler.

If you'd like, I can start with steps 1 and 2, since the later steps build on them.

Please start with steps 1 and 2. Please ask if there are parts of the structure you are unsure about.

Thought process

Thought process

Ran 18 commands

Ran 18 commands

Now SimpleJson and HardwareInfo.

Ran a command

Ran a command

Now MainVascular — read it.

Ran 3 commands

Ran 3 commands

Add missing accessors to LlamaServerConfig; adapt RunRecorder names.

Ran 11 commands, read a file

Ran 11 commands, read a file

Steps 1 and 2 are written. I haven't compiled everything, and I haven't run the program against a real llama-server.

What I checked. The settings, run-recording and hardware classes compiled and ran in my sandbox. I fed them fake settings files and a fake document. They produced the layered overrides, one combined error listing for bad values, and a complete run folder. runs.csv and summary.json came out correctly. MainVascular, LlamaVascularExtractor and VascularTsvWriter need Jackson and newer Java syntax that my sandbox lacks. I could only check that their braces balance. Build them in Eclipse and send me any compile errors.

Choosing the files.

- **Machine file:** this is unchanged (--config=, -Dvascular.config, or VASCULAR_CONFIG).
- **Experiment file:** add --experiment=PATH, -Dvascular.experiment=PATH, or VASCULAR_EXPERIMENT. A key in the experiment file overrides the same key in the machine file.
- **Experiment mode:** giving an experiment file turns it on. There are no console prompts and each run gets a fresh folder. Without an experiment file, the program behaves as before.

Run folder. Each run creates <output.base-directory>/<run_id>/, where run_id is the timestamp plus the machine, model and label names. It contains:

- **manifest.json:** hardware, llama-server version and command line, model file and size, prompt hash and whether it matches the built-in prompt, all settings in effect, and a corpus hash.
- **Result files:** results.json and four TSVs. Every row has a run_id column, and documents.tsv also has per-document wall time, request time, token counts, prompt and generation tokens per second, and start and end times.
- **Telemetry and logs:** summary.json, raw_responses/<doc>.txt (always saved), server.log and the program log.

- **Copies for later:** prompt.txt, copies of both settings files, corpus_manifest.tsv with per-document hashes, and a copy of the document list if you used one.

<base>/runs.csv gets one row per run, with the model, prompt hash, key server settings, document counts, load time, documents per minute, median and 95th-percentile latency, and token throughput. I used no accuracy code, as you asked.

New settings keys.

- **Prompt:** prompt.file, prompt.id.
- **Sampling:** sampling.temperature, sampling.top-k, sampling.top-p, sampling.repeat-penalty, sampling.seed.
- **Generation:** generation.max-tokens, generation.retry-max-tokens, generation.repair-max-tokens, generation.disable-thinking, generation.use-grammar.
- **Context budget:** budget.chars-per-token, budget.prompt-overhead-tokens, budget.safety-margin-tokens.
- **Experiment:** experiment.label, experiment.notes, experiment.model-label, experiment.document-list, experiment.hash-model-file.
- **Machine file:** output.base-directory, run.machine-name, run.hardware-notes.

The three machine samples now include the machine-file keys. Edit the placeholder paths and hardware notes in them. experiment-baseline.properties and experiment-prompt-variant.properties are new samples.

Using your own prompt. prompt-builtin.txt is the current built-in prompt as plain text. Copy it, edit it, and point prompt.file at the copy. Check that the baseline run's manifest says matches_builtin: true. If it doesn't, my conversion of the Java text block isn't exact, so tell me. The JSON schema and grammar stay in code, so a prompt edit must keep the same field names and allowed values.

Not done.

- **Resume of a named run folder:** you asked for this, and I left it out. Experiment runs are always fresh. Say so if you want it added.
- **Keep run folders on VA-approved storage.** They contain report-derived text in the source text and raw responses.

- **Steps 3 to 5** (batch runner, per-model adapter settings, resource sampler) are not started.

Files are in the outputs folder:

- AppSettings.java
- ExtractionOptions.java
- DocumentTelemetry.java
- RunRecorder.java
- SimpleJson.java
- HardwareInfo.java
- ContextBudget.java
- LlamaServerConfig.java
- LlamaVascularExtractor.java
- VascularTsvWriter.java
- MainVascular.java
- VascularThroughputHarness.java
- experiment-baseline.properties
- experiment-prompt-variant.properties
- prompt-builtin.txt
- vascular-va-xeon-a2.properties
- vascular-small-pc-cpu.properties
- vascular-mac-studio-m2max.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\AppSettings.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\ExtractionOptions.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\DocumentTelemetry.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\RunRecorder.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\SimpleJson.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\HardwareInfo.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\ContextBudget.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaServerConfig.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaVascularExtractor.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularTsvWriter.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\MainVascular.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularThroughputHarness.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\experiment-baseline.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\experiment-prompt-variant.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\prompt-builtin.txt

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\vascular-va-xeon-a2.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\vascular-small-pc-cpu.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\vascular-mac-studio-m2max.properties